

Frontend API Handoff Document

Laravel backend API documentation for the Healthy Menu API / healthy-menu-api project. Prepared for the frontend dashboard developer.

Generated: 2026-06-22

Source basis: uploaded Laravel project archive and verified chat execution outputs. Any unclear or unimplemented item is marked as Not confirmed or Not implemented.

1. Project Overview

Item	Confirmed Information
Project name	Healthy Menu API / healthy-menu-api
Backend purpose	Laravel RESTful API backend for a healthy restaurant/menu system. Backend APIs only; no frontend implementation.
Frontend/dashboard purpose	Frontend consumes public menu APIs and protected admin dashboard APIs to manage products, categories, ingredients, goals/tags, and product images.
Main modules	Authentication/login, Products, Categories, Ingredients, Goals/Tags, Product Images, Swagger/OpenAPI documentation.
API style	RESTful JSON APIs using Laravel controllers, Form Requests, API Resources, Services, and Eloquent relationships.
Database used	MySQL. Database name confirmed in handover: healthy_menu_api.
Authentication type	Laravel Sanctum personal access tokens, sent as Bearer tokens. Admin-only login; no public registration.
Current backend status	Core backend implemented and verified: Swagger route coverage passed, Pint passed, PHPStan passed, runtime smoke tests passed for public and protected endpoints.

2. Base API Information

Item	Value
Local development base URL	http://127.0.0.1:8000
Production/staging base URL	Not confirmed
API prefix	/api
API version in URL	Not implemented / Not used. Swagger title version is 1.0.0, but URLs do not include /v1.
Content type for JSON requests	application/json
Accept header	application/json
Auth header format	Authorization: Bearer {access_token}
Date/time format	ISO 8601 UTC strings, for example 2026-06-21T19:47:31.000000Z. Resources use toISOString() where timestamps are returned.
JSON naming convention	snake_case fields such as category_id, is_available, ingredient_ids, goal_ids.
File upload content type	multipart/form-data for product image upload.

```
Authorization: Bearer {access_token}
Accept: application/json
Content-Type: application/json
```

```
For image upload:
Content-Type: multipart/form-data
Field name: image
```

3. Authentication

Authentication is admin-only. There is no public registration endpoint. Non-admin users cannot log in through the admin login flow because AuthService checks is_admin before issuing a token.

Login

Item	Details
Purpose	Authenticates an admin user and returns a Sanctum access token.
Method	POST
URL	/api/auth/login
Auth required	No
Headers	Accept: application/json; Content-Type: application/json
Request body	email, password, optional device_name
Validation rules	email: required, email, max:255. password: required, string, min:8, max:100. device_name: nullable, string, max:100.
Possible errors	401 AUTH.INVALID_CREDENTIALS for wrong email/password/non-admin. 422 VALIDATION.ERROR for invalid request body.
Frontend notes	Store data.token securely on the frontend and attach it to protected admin requests as Authorization: Bearer {token}. Do not depend on the email/password error to distinguish which field failed; the backend intentionally returns the same message.

Response example:

```
{
  "success": true,
  "message": "Logged in successfully.",
  "data": {
    "token": "1|plain-text-token-example",
```

```

    "token_type": "Bearer",
    "user": {
      "id": 1,
      "name": "System Admin",
      "email": "admin@healthy-menu.test"
    }
  }
}

```

Logout

Item	Details
Purpose	Revokes the current authenticated Sanctum access token.
Method	POST
URL	/api/auth/logout
Auth required	Yes, auth:sanctum
Headers	Authorization: Bearer {access_token}; Accept: application/json
Request body	None
Validation rules	No request body validation.
Possible errors	401 AUTH.UNAUTHENTICATED if token is missing/invalid.
Frontend notes	After successful logout, remove the token from frontend storage and redirect user to login.

Response example:

```

{
  "success": true,
  "message": "Logged out successfully.",
  "data": null
}

```

Auth Item	Status	Notes
Current user/profile endpoint	Not implemented	No /api/auth/me or /api/profile endpoint is confirmed. Login response includes user id/name/email.
Token expiry behavior	Not confirmed	Sanctum token expiration policy was not confirmed in the project context. Frontend should handle 401 by clearing token and redirecting to login.
Public registration	Not implemented by design	Only seeded/admin users can log in.
Auth health endpoint	Implemented	GET /api/auth/health exists and is public; mainly useful for diagnostics.

4. Global Response Format

The backend uses App\Support\ApiResponse for consistent JSON responses.

Success response

```

{
  "success": true,
  "message": "Success",
  "data": {}
}

```

Single item response

```

{
  "success": true,
  "message": "Category retrieved successfully.",
  "data": {
    "id": 1,
    "name": "Bowls",
    "description": "Healthy protein bowls with balanced macros."
  }
}

```

List response

```

{
  "success": true,
  "message": "Goals retrieved successfully.",
  "data": [
    {
      "id": 1,

```

```
    "name": "High Protein"
  }
]
}
```

Paginated response

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "data": [],
  "meta": {
    "pagination": {
      "current_page": 1,
      "per_page": 10,
      "total": 0,
      "last_page": 1,
      "from": null,
      "to": null
    }
  }
}
```

Validation error response

```
{
  "success": false,
  "message": "Validation failed.",
  "code": "VALIDATION.ERROR",
  "errors": {
    "title": [
      "The title field is required."
    ]
  }
}
```

Unauthorized response

```
{
  "success": false,
  "message": "Unauthenticated.",
  "code": "AUTH.UNAUTHENTICATED"
}
```

Forbidden response

```
{
  "success": false,
  "message": "You are not allowed to access this resource.",
  "code": "AUTH.FORBIDDEN"
}
```

Not found response

```
{
  "success": false,
  "message": "Resource not found.",
  "code": "RESOURCE.NOT_FOUND"
}
```

Server error response

Exact production server error response shape is Not confirmed. Frontend should treat 5xx responses as generic backend/server failures and avoid showing raw technical details to users.

5. Global Validation and Error Handling

Status Code	Meaning	Backend Behavior / Frontend Handling
200	Success	Returned for normal successful GET/PUT/DELETE/logout operations.
201	Created	Returned by create endpoints and actual product image upload controller.

401	Unauthenticated	Missing/invalid token or invalid login credentials. Clear token and redirect to login for protected routes.
403	Forbidden	Authenticated user is not allowed, especially non-admin users hitting admin routes.
404	Not found	ModelNotFoundException is rendered as RESOURCE.NOT_FOUND for API requests. Product image delete also aborts 404 if product/image is missing.
409	Business conflict	Category/ingredient/goal delete is blocked if related products exist. Codes: CATEGORY.IN_USE, INGREDIENT.IN_USE, GOAL.IN_USE.
422	Validation failed	FormRequest validation errors. errors object maps fields to arrays of messages.
500	Server error	Exact response shape Not confirmed. Frontend should show a safe generic message.

- Validation errors always use message Validation failed. and code VALIDATION.ERROR.
- Authentication exceptions for /api/* are rendered as JSON 401, not HTML redirects.
- Not found model errors are rendered as JSON 404 with RESOURCE.NOT_FOUND.
- Uploaded images must be real image files, not just renamed files; Laravel image/file validation is used.

6. Pagination, Filtering, Searching, and Sorting

Endpoint	Pagination	Search	Filters	Sorting	Defaults
/api/menu/products	page min 1; per_page min 1 max 50	search title, description, ingredient name; max 100 chars	category_id exists categories; goal_id exists goals; min_protein 0..999.99; max_calories 0..9999; max_carbs 0..999.99; is_available true/false/1/0	newest, price_asc, price_desc, calories_asc, protein_desc	page=1, per_page=12, sort=newest
/api/admin/products	page min 1; per_page min 1 max 50	search title, description, ingredient name; max 100 chars	category_id, goal_id, ingredient_id, is_available true/false/1/0	newest, oldest, title_asc, price_asc, price_desc, protein_desc, calories_asc	page=1, per_page=10, sort=newest
/api/admin/categories	page min 1; per_page min 1 max 50	search name or description; max 100 chars	None confirmed	Ordered by name in service	page=1, per_page=10
/api/admin/ingredients	page min 1; per_page min 1 max 50	search name; max 100 chars	None confirmed	Ordered by name in service	page=1, per_page=10
/api/admin/goals	page min 1; per_page min 1 max 50	search name; max 100 chars	None confirmed	Ordered by name in service	page=1, per_page=10
/api/menu/categories, /api/menu/goals, /api/menu/ingredients	Not implemented	Not implemented	Not implemented	Ordered by name	No pagination

```
GET
/api/menu/products?search=chicken&category_id=1&goal_id=1&min_protein=30&max_calories=600&is_available=true&sort=protein_desc&page=1&per_page=12
GET /api/admin/products?ingredient_id=1&sort=title_asc&page=1&per_page=10
GET /api/admin/categories?search=bowl&page=1&per_page=10
```

7. Products API

Product APIs include public read-only menu endpoints and protected admin CRUD endpoints. The database column is carbs and the API response nests macros under nutrition. The user-facing word carb should map to the API field carbs.

Field	Type	Where Returned	Notes
id	integer	Public/admin product responses	Product primary key.
title	string	All product responses	max 180 on create/update.
description	string	Details/admin responses	Public product cards return short_description instead.
short_description	string	Public product list only	Generated with Str::limit(description, 120).
price	number	All product responses	decimal stored as price(10,2), returned as float.
nutrition.protein	number	All product responses	Maps product protein column.
nutrition.carbs	number	All product responses	Maps product carbs column.
nutrition.fat	number	All product responses	Maps product fat column.
nutrition.calories	integer	All product responses	Maps product calories column.
category_id	integer	Admin product resource	Required on create/update.
is_available	boolean	All product responses	Frontend should use this to show available/unavailable state.
category	object	Public/admin product responses when eager loaded	CategoryResource.
ingredients	array	Public details and admin show/create/update when loaded	Not returned in public card list. Admin list returns ingredients_count, not full ingredients.
ingredient_ids	array<int>	Admin show/create/update when ingredients loaded	Frontend should send ingredient_ids array on create/update.
goals	array	Public/admin product responses	Goals/tags/badges.
goal_ids	array<int>	Admin product resource when goals loaded	Frontend should send goal_ids array on create/update.
image	object null	Product responses when image eager loaded	Single product image only.
created_at / updated_at	string	Admin product resource	ISO 8601 UTC.

Public list products

Item	Details
Purpose	Returns paginated public product cards.
Method	GET
URL	/api/menu/products
Auth required	No
Headers	Accept: application/json
Query parameters	search, category_id, goal_id, min_protein, max_calories, max_carbs, is_available, sort, page, per_page
Validation rules	See section 6. Public default page=1, per_page=12, sort=newest.
Possible errors	422 validation error for invalid query parameters.
Frontend notes	Use for public menu cards. It does not include full ingredients; fetch details by id for modal/full view.

Response example:

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "data": [
    {
      "id": 1,
      "title": "Chicken Protein Bowl",
      "short_description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
      "price": 7.5,
      "nutrition": {
        "protein": 45,
        "carbs": 55,
        "fat": 12,
        "calories": 520
      },
      "is_available": true,
      "category": {
        "id": 1,
        "name": "Bowls",
        "description": "Healthy protein bowls with balanced macros."
      },
      "goals": [
        {
          "id": 1,
          "name": "High Protein"
        }
      ],
      "image": null
    }
  ],
  "meta": {
    "pagination": {
      "current_page": 1,
      "per_page": 12,
      "total": 1,
      "last_page": 1,
      "from": 1,
      "to": 1
    }
  }
}
```

Public get product by id

Item	Details
Purpose	Returns full product details for the public product details modal.
Method	GET
URL	/api/menu/products/{id}
Auth required	No
Headers	Accept: application/json
Route parameters	id: integer, numeric route constraint.
Validation rules	id must be numeric due route constraint.
Possible errors	404 RESOURCE.NOT_FOUND if product does not exist.
Frontend notes	Use this when opening product details modal.

Response example:

```
{
  "success": true,
  "message": "Product retrieved successfully.",
  "data": {
    "id": 1,
```

```
{
  "title": "Chicken Protein Bowl",
  "description": "Full product description.",
  "price": 7.5,
  "nutrition": {
    "protein": 45,
    "carbs": 55,
    "fat": 12,
    "calories": 520
  },
  "is_available": true,
  "category": {
    "id": 1,
    "name": "Bowls"
  },
  "ingredients": [
    {
      "id": 1,
      "name": "Chicken Breast"
    }
  ],
  "goals": [
    {
      "id": 1,
      "name": "High Protein"
    }
  ],
  "image": null
}
```

Admin list products

Item	Details
Purpose	Returns paginated admin product list with filters, counts, category/goals/image.
Method	GET
URL	/api/admin/products
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Accept: application/json
Query parameters	search, category_id, goal_id, ingredient_id, is_available, sort, page, per_page
Validation rules	See section 6. Admin default page=1, per_page=10, sort=newest.
Possible errors	401, 403, 422.
Frontend notes	Admin list may not include the full ingredients array because list query loads category, goals, image, and counts. Fetch by id for full edit payload.

Response example:

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "data": [
    {
      "id": 1,
      "title": "Chicken Protein Bowl",
      "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
      "price": 7.5,
      "nutrition": {
        "protein": 45,
        "carbs": 55,
        "fat": 12,
        "calories": 520
      },
      "category_id": 1,
      "is_available": true,
      "category": {
        "id": 1,
        "name": "Bowls",
        "description": "Healthy protein bowls with balanced macros."
      },
      "ingredients": [
        {
          "id": 1,
          "name": "Chicken Breast"
        }
      ],
      "ingredient_ids": [
        1,
        6,
        11,
        12
      ]
    }
  ]
}
```

```

    ],
    "goals": [
      {
        "id": 1,
        "name": "High Protein"
      },
      {
        "id": 5,
        "name": "Performance"
      }
    ],
    "goal_ids": [
      1,
      5
    ],
    "ingredients_count": 4,
    "goals_count": 2,
    "image": null,
    "created_at": "2026-06-21T20:30:50.000000Z",
    "updated_at": "2026-06-21T20:30:50.000000Z"
  }
],
"meta": {
  "pagination": {
    "current_page": 1,
    "per_page": 10,
    "total": 1,
    "last_page": 1,
    "from": 1,
    "to": 1
  }
}
}
}

```

Admin get product by id

Item	Details
Purpose	Returns one product with category, ingredients, goals, image, and counts.
Method	GET
URL	/api/admin/products/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Accept: application/json
Route parameters	id: integer, numeric route constraint.
Possible errors	401, 403, 404.
Frontend notes	Use before editing a product to prefill category id, ingredient ids, goal ids, nutrition, availability, and current image.

Response example:

```

{
  "success": true,
  "message": "Product retrieved successfully.",
  "data": {
    "id": 1,
    "title": "Chicken Protein Bowl",
    "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
    "price": 7.5,
    "nutrition": {
      "protein": 45,
      "carbs": 55,
      "fat": 12,
      "calories": 520
    },
    "category_id": 1,
    "is_available": true,
    "category": {
      "id": 1,
      "name": "Bowls",
      "description": "Healthy protein bowls with balanced macros."
    },
    "ingredients": [
      {
        "id": 1,
        "name": "Chicken Breast"
      }
    ],
    "ingredient_ids": [
      1,
      6,
      11,

```



```

    12
  ],
  "goals": [
    {
      "id": 1,
      "name": "High Protein"
    },
    {
      "id": 5,
      "name": "Performance"
    }
  ],
  "goal_ids": [
    1,
    5
  ],
  "ingredients_count": 4,
  "goals_count": 2,
  "image": null,
  "created_at": "2026-06-21T20:30:50.000000Z",
  "updated_at": "2026-06-21T20:30:50.000000Z"
}
}

```

Admin create product

Item	Details
Purpose	Creates a product and syncs ingredient and goal relationships in a database transaction.
Method	POST
URL	/api/admin/products
Auth required	Yes
Headers	Authorization: Bearer {access token}; Accept: application/json; Content-Type: application/json
Request body	<pre> { "title": "Chicken Protein Bowl", "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.", "price": 7.5, "protein": 45, "carbs": 55, "fat": 12, "calories": 520, "category_id": 1, "is_available": true, "ingredient_ids": [1, 6, 11, 12], "goal_ids": [1, 5] } </pre>
Validation rules	title required string max:180; description required string max:5000; price required numeric 0..999999.99; protein/carbs/fat required numeric 0..999.99; calories required integer 0..9999; category_id required integer exists; is_available required boolean; ingredient_ids required array min:1; each ingredient id required integer distinct exists; goal_ids required array min:1; each goal id required integer distinct exists.
Possible errors	401, 403, 422.
Frontend notes	Send ingredient_ids and goal_ids as arrays of existing IDs. Product image is uploaded separately after product creation.

Response example:

```

{
  "success": true,
  "message": "Product created successfully.",
  "data": {
    "id": 1,
    "title": "Chicken Protein Bowl",
    "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
    "price": 7.5,
    "nutrition": {
      "protein": 45,
      "carbs": 55,
      "fat": 12,
      "calories": 520
    },
    "category_id": 1,
    "is_available": true,
    "category": {
      "id": 1,

```

```
    "name": "Bowls",
    "description": "Healthy protein bowls with balanced macros."
  },
  "ingredients": [
    {
      "id": 1,
      "name": "Chicken Breast"
    }
  ],
  "ingredient_ids": [
    1,
    6,
    11,
    12
  ],
  "goals": [
    {
      "id": 1,
      "name": "High Protein"
    },
    {
      "id": 5,
      "name": "Performance"
    }
  ],
  "goal_ids": [
    1,
    5
  ],
  "ingredients_count": 4,
  "goals_count": 2,
  "image": null,
  "created_at": "2026-06-21T20:30:50.000000Z",
  "updated_at": "2026-06-21T20:30:50.000000Z"
}
}
```

Admin update product

Item	Details
Purpose	Updates an existing product and syncs ingredient and goal relationships. This is full PUT behavior.
Method	PUT
URL	/api/admin/products/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Accept: application/json; Content-Type: application/json
Route parameters	id: integer, numeric route constraint.
Request body	{ "title": "Updated Chicken Protein Bowl", "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.", "price": 7.5, "protein": 45, "carbs": 55, "fat": 12, "calories": 520, "category_id": 1, "is_available": true, "ingredient_ids": [1, 6, 11, 12], "goal_ids": [1, 5] }
Validation rules	Same as create product. All fields are required.
Possible errors	401, 403, 404, 422.
Frontend notes	Because it is full PUT, do not send partial payloads. Always send complete product form data.

Response example:

```
{
  "success": true,
  "message": "Product updated successfully.",
  "data": {
    "id": 1,
    "title": "Chicken Protein Bowl",
    "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
```

```

"price": 7.5,
"nutrition": {
  "protein": 45,
  "carbs": 55,
  "fat": 12,
  "calories": 520
},
"category_id": 1,
"is_available": true,
"category": {
  "id": 1,
  "name": "Bowls",
  "description": "Healthy protein bowls with balanced macros."
},
"ingredients": [
  {
    "id": 1,
    "name": "Chicken Breast"
  }
],
"ingredient_ids": [
  1,
  6,
  11,
  12
],
"goals": [
  {
    "id": 1,
    "name": "High Protein"
  },
  {
    "id": 5,
    "name": "Performance"
  }
],
"goal_ids": [
  1,
  5
],
"ingredients_count": 4,
"goals_count": 2,
"image": null,
"created_at": "2026-06-21T20:30:50.000000Z",
"updated_at": "2026-06-21T20:30:50.000000Z"
}
}

```

Admin delete product

Item	Details
Purpose	Deletes a product and then deletes its physical image folder under products/{id}.
Method	DELETE
URL	/api/admin/products/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Accept: application/json
Route parameters	id: integer, numeric route constraint.
Possible errors	401, 403, 404.
Frontend notes	After delete, remove product from frontend state. Backend deletes physical image directory from public disk.

Response example:

```

{
  "success": true,
  "message": "Product deleted successfully.",
  "data": null
}

```

Product-related endpoint not available	Status	Notes
Availability/status-only endpoint	Not implemented	Use PUT /api/admin/products/{id} with full payload to change is_available.
Attach/detach individual ingredient endpoint	Not implemented	Use full product update with ingredient_ids array.
Attach/detach individual goal endpoint	Not implemented	Use full product update with goal_ids array.

8. Categories API

Field	Type	Notes
id	integer	Primary key
name	string	required unique max 120
description	string null	nullable max 2000
products_count	integer	Admin list/show when counted
created_at / updated_at	string	ISO 8601 UTC where returned

Public list categories

Item	Details
Purpose	Returns all public categories ordered by name.
Method	GET
URL	/api/menu/categories
Auth required	No
Headers	Accept: application/json
Frontend notes	Use as public menu filter lookup.

Response example:

```
{
  "success": true,
  "message": "Categories retrieved successfully.",
  "data": [
    {
      "id": 1,
      "name": "Bowls",
      "description": "Healthy protein bowls with balanced macros.",
      "created_at": "2026-06-21T19:47:31.000000Z",
      "updated_at": "2026-06-21T19:47:31.000000Z"
    }
  ]
}
```

Admin list categories

Item	Details
Purpose	Returns paginated categories with products_count.
Method	GET
URL	/api/admin/categories
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Query parameters	search, page, per_page
Validation rules	search nullable string max:100; page nullable integer min:1; per_page nullable integer min:1 max:50. Defaults page=1, per_page=10.
Possible errors	401, 403, 422.

Response example:

```
{
  "success": true,
  "message": "Categories retrieved successfully.",
  "data": [
    {
      "id": 1,
      "name": "Bowls",
      "description": "Healthy protein bowls with balanced macros.",
      "products_count": 1,
      "created_at": "2026-06-21T19:47:31.000000Z",
      "updated_at": "2026-06-21T19:47:31.000000Z"
    }
  ],
  "meta": {
    "pagination": {
      "current_page": 1,
      "per_page": 10,
      "total": 6,
      "last_page": 1,
      "from": 1,
      "to": 6
    }
  }
}
```

Admin get category by id

Item	Details
Purpose	Returns one category with products_count.
Method	GET
URL	/api/admin/categories/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Route parameters	id integer
Possible errors	401, 403, 404.

Response example:

```
{
  "success": true,
  "message": "Category retrieved successfully.",
  "data": {
    "id": 1,
    "name": "Bowls",
    "description": "Healthy protein bowls with balanced macros.",
    "products_count": 1
  }
}
```

Admin create category

Item	Details
Purpose	Creates a new category.
Method	POST
URL	/api/admin/categories
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Content-Type: application/json
Request body	name, optional description
Validation rules	name required string max:120 unique:categories,name. description nullable string max:2000.
Possible errors	401, 403, 422.

Response example:

```
{
  "success": true,
  "message": "Category created successfully.",
  "data": {
    "id": 7,
    "name": "Smoothies",
    "description": "Healthy smoothie options."
  }
}
```

Admin update category

Item	Details
Purpose	Updates a category.
Method	PUT
URL	/api/admin/categories/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Content-Type: application/json
Route parameters	id integer
Request body	name, optional description
Validation rules	name required string max:120 unique except current id. description nullable string max:2000.
Possible errors	401, 403, 404, 422.

Response example:

```
{
  "success": true,
  "message": "Category updated successfully.",
  "data": {
    "id": 1,
    "name": "Protein Bowls",
    "description": "Updated description.",
    "products_count": 1
  }
}
```

Admin delete category

Item	Details
Purpose	Deletes category only if it has no products.
Method	DELETE
URL	/api/admin/categories/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Route parameters	id integer
Possible errors	401, 403, 404, 409 CATEGORY.IN_USE.
Frontend notes	Delete is blocked when products exist. Database also restricts product.category_id delete; service returns 409 before DB constraint failure.

Response example:

```
{
  "success": true,
  "message": "Category deleted successfully.",
  "data": null
}
```

9. Ingredients API

Field	Type	Notes
id	integer	Primary key
name	string	required unique max 120
products_count	integer	Admin list/show when counted
created_at / updated_at	string	ISO 8601 UTC where returned

Public list ingredients

Item	Details
Purpose	Returns all ingredients ordered by name.
Method	GET
URL	/api/menu/ingredients
Auth required	No
Headers	Accept: application/json
Frontend notes	Use as lookup data for product details/filter UI if needed.

Response example:

```
{
  "success": true,
  "message": "Ingredients retrieved successfully.",
  "data": [
    {
      "id": 1,
      "name": "Chicken Breast",
      "created_at": "2026-06-21T19:47:31.000000Z",
      "updated_at": "2026-06-21T19:47:31.000000Z"
    }
  ]
}
```

Admin list ingredients

Item	Details
Purpose	Returns paginated ingredients with products_count.
Method	GET
URL	/api/admin/ingredients
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Query parameters	search, page, per_page
Validation rules	search nullable string max:100; page nullable integer min:1; per_page nullable integer min:1 max:50. Defaults page=1, per_page=10.
Possible errors	401, 403, 422.

Response example:

```
{
  "success": true,
  "message": "Ingredients retrieved successfully.",
  "data": [
    {
      "id": 1,
      "name": "Chicken Breast",
      "products_count": 1
    }
  ],
  "meta": {
    "pagination": {
      "current_page": 1,
      "per_page": 10,
      "total": 20,
      "last_page": 2,
      "from": 1,
      "to": 10
    }
  }
}
```

Admin get ingredient by id

Item	Details
Purpose	Returns one ingredient with products_count.
Method	GET
URL	/api/admin/ingredients/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Route parameters	id integer
Possible errors	401, 403, 404.

Response example:

```
{
  "success": true,
  "message": "Ingredient retrieved successfully.",
  "data": {
    "id": 1,
    "name": "Chicken Breast",
    "products_count": 1
  }
}
```

Admin create ingredient

Item	Details
Purpose	Creates a new ingredient.
Method	POST
URL	/api/admin/ingredients
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Content-Type: application/json
Request body	name
Validation rules	name required string max:120 unique:ingredients.name.
Possible errors	401, 403, 422.

Response example:

```
{
  "success": true,
  "message": "Ingredient created successfully.",
  "data": {
    "id": 21,
    "name": "Blueberries"
  }
}
```

Admin update ingredient

Item	Details
Purpose	Updates an ingredient.
Method	PUT
URL	/api/admin/ingredients/{id}

Auth required	Yes
Headers	Authorization: Bearer {access_token}; Content-Type: application/json
Route parameters	id integer
Request body	name
Validation rules	name required string max:120 unique except current id.
Possible errors	401, 403, 404, 422.

Response example:

```
{
  "success": true,
  "message": "Ingredient updated successfully.",
  "data": {
    "id": 1,
    "name": "Grilled Chicken Breast",
    "products_count": 1
  }
}
```

Admin delete ingredient

Item	Details
Purpose	Deletes ingredient only if no products use it.
Method	DELETE
URL	/api/admin/ingredients/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Route parameters	id integer
Possible errors	401, 403, 404, 409 INGREDIENT.IN_USE.
Frontend notes	Ingredients attach to products through ingredient_ids array on product create/update. There is no separate attach/detach endpoint.

Response example:

```
{
  "success": true,
  "message": "Ingredient deleted successfully.",
  "data": null
}
```

10. Goals API

Goals are used as product goals/tags/badges such as High Protein, Low Carb, Bulking, Cutting, Performance, Bestseller, Omega-3.

Field	Type	Notes
id	integer	Primary key
name	string	required unique max 120
products_count	integer	Admin list/show when counted
created_at / updated_at	string	ISO 8601 UTC where returned

Public list goals

Item	Details
Purpose	Returns all public goals/tags ordered by name.
Method	GET
URL	/api/menu/goals
Auth required	No
Headers	Accept: application/json
Frontend notes	Use for product card badges/filter UI.

Response example:

```
{
  "success": true,
  "message": "Goals retrieved successfully.",
  "data": [
    {
      "id": 1,
      "name": "High Protein",

```



```

    "created_at": "2026-06-21T19:47:31.000000Z",
    "updated_at": "2026-06-21T19:47:31.000000Z"
  }
}

```

Admin list goals

Item	Details
Purpose	Returns paginated goals with products_count.
Method	GET
URL	/api/admin/goals
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Query parameters	search, page, per_page
Validation rules	search nullable string max:100; page nullable integer min:1; per_page nullable integer min:1 max:50. Defaults page=1, per_page=10.
Possible errors	401, 403, 422.

Response example:

```

{
  "success": true,
  "message": "Goals retrieved successfully.",
  "data": [
    {
      "id": 1,
      "name": "High Protein",
      "products_count": 1
    }
  ],
  "meta": {
    "pagination": {
      "current_page": 1,
      "per_page": 10,
      "total": 7,
      "last_page": 1,
      "from": 1,
      "to": 7
    }
  }
}

```

Admin get goal by id

Item	Details
Purpose	Returns one goal with products_count.
Method	GET
URL	/api/admin/goals/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Route parameters	id integer
Possible errors	401, 403, 404.

Response example:

```

{
  "success": true,
  "message": "Goal retrieved successfully.",
  "data": {
    "id": 1,
    "name": "High Protein",
    "products_count": 1
  }
}

```

Admin create goal

Item	Details
Purpose	Creates a new goal/tag.
Method	POST
URL	/api/admin/goals
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Content-Type: application/json
Request body	name

Validation rules	name required string max:120 unique:goals,name.
Possible errors	401, 403, 422.

Response example:

```
{
  "success": true,
  "message": "Goal created successfully.",
  "data": {
    "id": 8,
    "name": "Low Sugar"
  }
}
```

Admin update goal

Item	Details
Purpose	Updates a goal/tag.
Method	PUT
URL	/api/admin/goals/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Content-Type: application/json
Route parameters	id integer
Request body	name
Validation rules	name required string max:120 unique except current id.
Possible errors	401, 403, 404, 422.

Response example:

```
{
  "success": true,
  "message": "Goal updated successfully.",
  "data": {
    "id": 1,
    "name": "High Protein",
    "products_count": 1
  }
}
```

Admin delete goal

Item	Details
Purpose	Deletes goal only if no products use it.
Method	DELETE
URL	/api/admin/goals/{id}
Auth required	Yes
Headers	Authorization: Bearer {access_token}
Route parameters	id integer
Possible errors	401, 403, 404, 409 GOAL.IN USE.
Frontend notes	Goals attach to products through goal_ids array on product create/update. There is no separate attach/detach endpoint.

Response example:

```
{
  "success": true,
  "message": "Goal deleted successfully.",
  "data": null
}
```

11. Product Images API

Final confirmed design is one image per product. Multiple images, set-primary image, image listing, and image metadata update endpoints are not implemented.

Stored Model Field	Returned in API?	Notes
id	Yes	Product image id.
product_id	No	Stored internally, not exposed by ProductImageResource.
file_size	Yes	Integer bytes from uploaded file size.

file_name	Yes	Original client filename.
extension	Yes	Lowercase extension.
path	No	Internal storage path is not exposed.
url	Yes	Absolute public URL generated by asset("storage/" . path).
created_at / updated_at	No	Stored by model but not returned by ProductImageResource.

Upload or replace product image

Item	Details
Purpose	Uploads one image. If image already exists, it is replaced. The old physical file is deleted after DB update succeeds.
Method	POST
URL	/api/admin/products/{productId}/image
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Accept: application/json; Content-Type: multipart/form-data
Route parameters	productId: integer, numeric route constraint.
Request body	multipart field image
Validation rules	image required, file, image, mimes:jpg,jpeg,png,webp, max:5120 KB. SVG is not allowed.
Possible errors	401, 403, 404 product not found, 422 invalid/missing/large/wrong MIME image.
Frontend notes	Actual controller returns HTTP 201. The current OpenAPI file in the uploaded project documents response 200 for upload; see Known Limitations.

Response example:

```
{
  "success": true,
  "message": "Product image uploaded successfully.",
  "data": {
    "id": 1,
    "file_name": "food.jpg",
    "extension": "jpg",
    "file_size": 11214,
    "url": "http://127.0.0.1:8000/storage/products/1/image.jpg"
  }
}
```

Delete product image

Item	Details
Purpose	Deletes the current image record and physical file.
Method	DELETE
URL	/api/admin/products/{productId}/image
Auth required	Yes
Headers	Authorization: Bearer {access_token}; Accept: application/json
Route parameters	productId: integer, numeric route constraint.
Possible errors	401, 403, 404 product or image not found.
Frontend notes	After delete, product responses return image: null. Product deletion also deletes the products/{productId} image directory.

Response example:

```
{
  "success": true,
  "message": "Product image deleted successfully.",
  "data": null
}
```

Image Capability	Status	Notes
List product images	Not implemented	Only one image is allowed.
Set primary image	Not implemented	is_primary was removed.
Update image metadata	Not implemented	Only upload/replace and delete exist.
Multiple images	Not implemented by design	product_images.product_id has a unique constraint.
Public URL behavior	Confirmed	API returns absolute URL from asset(storage/path). Frontend should render url directly.

12. Dashboard Integration Flow

1. Open the admin login page and POST email/password/device_name to /api/auth/login.
2. Store data.token from login response and attach Authorization: Bearer {token} to protected admin calls.

- Load lookup data for forms: GET /api/admin/categories, GET /api/admin/ingredients, GET /api/admin/goals. Public lookup endpoints can also be used where auth is not desired.
- Load products list with GET /api/admin/products?page=1&per_page=10 and optional filters.
- For product create form, send full JSON payload to POST /api/admin/products with category_id, nutrition fields, ingredient_ids, goal_ids, and is_available.
- After product creation, upload the product image using POST /api/admin/products/{productId}/image with multipart/form-data field image.
- For edit form, GET /api/admin/products/{id} first to prefill fields and selected ingredient_ids/goal_ids, then PUT the full payload to /api/admin/products/{id}.
- For replacing image, call POST /api/admin/products/{productId}/image again. Existing image is replaced.
- For deleting a product, call DELETE /api/admin/products/{id}. The backend deletes the product and its image folder.
- For logout, call POST /api/auth/logout and remove token from frontend storage.

13. Endpoint Summary Table

Module	Method	Endpoint	Auth	Purpose	Status
Health	GET	/api/menu/health	No	Public menu API health check	Implemented; Swagger documented
Health	GET	/api/auth/health	No	Auth API health check	Implemented; Swagger documentation Not confirmed
Health	GET	/api/admin/health	Yes	Protected admin API health check	Implemented; Swagger documentation Not confirmed
Authentication	POST	/api/auth/login	No	Admin login	Implemented; Swagger documented
Authentication	POST	/api/auth/logout	Yes	Logout/revoke token	Implemented; Swagger documented
Public Menu	GET	/api/menu/categories	No	Public categories lookup	Implemented; Swagger documented
Public Menu	GET	/api/menu/goals	No	Public goals/tags lookup	Implemented; Swagger documented
Public Menu	GET	/api/menu/ingredients	No	Public ingredients lookup	Implemented; Swagger documented
Public Products	GET	/api/menu/products	No	Public product cards	Implemented; Swagger documented
Public Products	GET	/api/menu/products/{id}	No	Public product details	Implemented; Swagger documented
Admin Categories	GET	/api/admin/categories	Yes	List categories	Implemented; Swagger documented
Admin Categories	POST	/api/admin/categories	Yes	Create category	Implemented; Swagger documented
Admin Categories	GET	/api/admin/categories/{id}	Yes	Get category	Implemented; Swagger documented
Admin Categories	PUT	/api/admin/categories/{id}	Yes	Update category	Implemented; Swagger documented
Admin Categories	DELETE	/api/admin/categories/{id}	Yes	Delete category	Implemented; Swagger documented
Admin Ingredients	GET	/api/admin/ingredients	Yes	List ingredients	Implemented; Swagger documented
Admin Ingredients	POST	/api/admin/ingredients	Yes	Create ingredient	Implemented; Swagger documented
Admin Ingredients	GET	/api/admin/ingredients/{id}	Yes	Get ingredient	Implemented; Swagger documented
Admin Ingredients	PUT	/api/admin/ingredients/{id}	Yes	Update ingredient	Implemented; Swagger documented
Admin Ingredients	DELETE	/api/admin/ingredients/{id}	Yes	Delete ingredient	Implemented; Swagger documented
Admin Goals	GET	/api/admin/goals	Yes	List goals	Implemented; Swagger documented
Admin Goals	POST	/api/admin/goals	Yes	Create goal	Implemented; Swagger documented
Admin Goals	GET	/api/admin/goals/{id}	Yes	Get goal	Implemented; Swagger documented
Admin Goals	PUT	/api/admin/goals/{id}	Yes	Update goal	Implemented; Swagger documented
Admin Goals	DELETE	/api/admin/goals/{id}	Yes	Delete goal	Implemented; Swagger documented
Admin Products	GET	/api/admin/products	Yes	List products	Implemented; Swagger documented
Admin Products	POST	/api/admin/products	Yes	Create product	Implemented; Swagger documented
Admin Products	GET	/api/admin/products/{id}	Yes	Get product	Implemented; Swagger documented
Admin Products	PUT	/api/admin/products/{id}	Yes	Update product	Implemented; Swagger documented
Admin Products	DELETE	/api/admin/products/{id}	Yes	Delete product	Implemented; Swagger documented
Admin Product Images	POST	/api/admin/products/{productId}/image	Yes	Upload/replace product image	Implemented; Swagger documented with status mismatch note
Admin Product Images	DELETE	/api/admin/products/{productId}/image	Yes	Delete product image	Implemented; Swagger documented

14. Request/Response Examples

Login request

```
{
  "email": "admin@healthy-menu.test",
  "password": "Admin@123456",
  "device_name": "dashboard"
}
```

Login success

```
{
  "success": true,
  "message": "Logged in successfully.",
  "data": {
    "token": "1|plain-text-token-example",
    "token_type": "Bearer",
    "user": {
      "id": 1,
      "name": "System Admin",
      "email": "admin@healthy-menu.test"
    }
  }
}
```

Create product request

```
{
  "title": "Chicken Protein Bowl",
  "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
  "price": 7.5,
  "protein": 45,
  "carbs": 55,
  "fat": 12,
  "calories": 520,
  "category_id": 1,
  "is_available": true,
  "ingredient_ids": [
    1,
    6,
    11,
    12
  ],
  "goal_ids": [
    1,
    5
  ]
}
```

Update product request

```
{
  "title": "Updated Chicken Protein Bowl",
  "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
  "price": 7.5,
  "protein": 45,
  "carbs": 55,
  "fat": 12,
  "calories": 520,
  "category_id": 1,
  "is_available": true,
  "ingredient_ids": [
    1,
    6,
    11,
    12
  ],
  "goal_ids": [
    1,
    5
  ]
}
```

Product list with relationships

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "data": [
    {
      "id": 1,
      "title": "Chicken Protein Bowl",
      "description": "Grilled chicken breast with rice, broccoli, avocado, and light fitness sauce.",
      "price": 7.5,
      "nutrition": {
        "protein": 45,
        "carbs": 55,
        "fat": 12,
        "calories": 520
      },
      "category_id": 1,
      "is_available": true,
      "category": {
        "id": 1,
        "name": "Bowls",
        "description": "Healthy protein bowls with balanced macros."
      },
      "ingredients": [
        {
          "id": 1,
          "name": "Chicken Breast"
        }
      ],
      "ingredient_ids": [
        1,
        6,
        11,
        12
      ],
      "goals": [
        {
          "id": 1,
          "name": "High Protein"
        },
        {
          "id": 5,
          "name": "Performance"
        }
      ],
      "goal_ids": [
        1,
        5
      ],
      "ingredients_count": 4,
      "goals_count": 2,
      "image": null,
      "created_at": "2026-06-21T20:30:50.000000Z",
      "updated_at": "2026-06-21T20:30:50.000000Z"
    }
  ],
  "meta": {
    "pagination": {
      "current_page": 1,
      "per_page": 10,
      "total": 1,
      "last_page": 1,
      "from": 1,
      "to": 1
    }
  }
}
```

Create category request

```
{
  "name": "Smoothies",
  "description": "Healthy smoothies and shakes."
}
```

Create ingredient request

```
{
  "name": "Blueberries"
}
```

```
}
```

Create goal request

```
{
  "name": "Low Sugar"
}
```

Upload image request

POST /api/admin/products/1/image
Content-Type: multipart/form-data
Authorization: Bearer {access_token}

Form field:
image = selected file

Upload image success

```
{
  "success": true,
  "message": "Product image uploaded successfully.",
  "data": {
    "id": 1,
    "file_name": "food.jpg",
    "extension": "jpg",
    "file_size": 11214,
    "url": "http://127.0.0.1:8000/storage/products/1/file.jpg"
  }
}
```

Validation error

```
{
  "success": false,
  "message": "Validation failed.",
  "code": "VALIDATION.ERROR",
  "errors": {
    "title": [
      "The title field is required."
    ],
    "ingredient_ids": [
      "The ingredient ids field is required."
    ]
  }
}
```

Unauthorized error

```
{
  "success": false,
  "message": "Unauthenticated.",
  "code": "AUTH.UNAUTHENTICATED"
}
```

15. Frontend Validation Guide

Login form

Field	Type	Required	Rules	Notes
email	string/email	Yes	email, max 255	Do not reveal whether email exists.
password	string	Yes	min 8, max 100	Use password input.
device_name	string	No	max 100	Optional; use dashboard/browser/device label.

Product create/edit form

Field	Type	Required	Constraints	Notes
-------	------	----------	-------------	-------

title	string	Yes	max 180	Product title.
description	string	Yes	max 5000	Full description. Public card short_description is generated backend-side.
price	number	Yes	0 to 999999.99	Use decimal input.
protein	number	Yes	0 to 999.99	grams.
carbs	number	Yes	0 to 999.99	API field is carbs, not carb.
fat	number	Yes	0 to 999.99	grams.
calories	integer	Yes	0 to 9999	integer only.
category_id	integer	Yes	must exist in categories	Load categories first.
is_available	boolean	Yes	true/false	Use switch/checkbox.
ingredient_ids	array<int>	Yes	min 1; ids distinct and must exist	Multi-select.
goal_ids	array<int>	Yes	min 1; ids distinct and must exist	Multi-select tags/badges.

Category form

Field	Type	Required	Constraints	Notes
name	string	Yes	max 120, unique	Show duplicate errors from backend.
description	string	No	max 2000	Nullable.

Ingredient form

Field	Type	Required	Constraints	Notes
name	string	Yes	max 120, unique	No description field implemented.

Goal form

Field	Type	Required	Constraints	Notes
name	string	Yes	max 120, unique	Goals are tags/badges.

Product image upload form

Field	Type	Required	Constraints	Notes
image	file	Yes	image file; mimes jpg,jpeg,png,webp; max 5120 KB	Use multipart/form-data. SVG not allowed. One image per product; upload again replaces old image.

16. Known Limitations / Not Confirmed Items

Item	Status	Notes
Production/staging base URL	Not confirmed	Only local development base URL http://127.0.0.1:8000 is confirmed.
Token expiry behavior	Not confirmed	No explicit token expiry policy was confirmed. Frontend must handle 401 globally.
Current user/profile endpoint	Not implemented	Use login response user object unless a profile endpoint is later added.
Public registration	Not implemented by design	Admin-only login.
Multiple product images	Not implemented by design	One image per product enforced by unique product_id.
Set primary image endpoint	Not implemented	is_primary was removed.
Product image list endpoint	Not implemented	Use image inside product responses.
Partial product update/PATCH	Not implemented	PUT requires complete payload.
Availability-only endpoint	Not implemented	Use full product PUT.
Postman collection	Not created / Not confirmed	Swagger/OpenAPI is available instead.
Automated tests	Not implemented / Not confirmed	Pint, PHPStan, Swagger coverage, and runtime smoke tests passed. PHPUnit feature tests are not confirmed.
Production deployment	Not covered	No production hosting/configuration handoff confirmed.
Swagger product image upload status	Needs correction	Actual controller returns 201 for upload. Current OpenAPI file in uploaded project documents response 200 for upload.
Auth/admin health Swagger docs	Not confirmed	Routes exist for /api/auth/health and /api/admin/health; Swagger coverage check only confirmed /api/menu/health plus main API endpoints.

17. Swagger/OpenAPI

Item	Confirmed Information
Swagger implemented	Yes. L5-Swagger/darkonline and swagger-php are used. Implementation uses PHP 8 attributes under app/OpenApi.
Swagger UI URL	http://127.0.0.1:8000/api/documentation
OpenAPI JSON	storage/api-docs/api-docs.json generated locally; public URL path depends on L5-Swagger config.
OpenAPI title/version	Healthy Menu API, version 1.0.0.
Server in OpenAPI	http://127.0.0.1:8000 local development server.
Security scheme	sanctum HTTP bearer token. Use Authorization: Bearer {token}.
Regenerate command	php artisan l5-swagger:generate

- Documented in Swagger: Public Menu, Authentication, Admin Categories, Admin Ingredients, Admin Goals, Admin Products, Admin Product Images.
- Swagger route coverage check confirmed all expected main paths are present in api-docs.json.
- To authenticate in Swagger UI, click Authorize and enter the Sanctum Bearer token according to the displayed scheme. If the UI does not prepend Bearer automatically, paste Bearer {token}.
- Important: actual product image upload controller returns 201, while the current OpenAPI file documents upload response 200. Frontend should trust the actual API behavior until Swagger is corrected.

```
php artisan optimize:clear
php artisan ls-swagger:generate

Open Swagger UI:
http://127.0.0.1:8000/api/documentation
```

18. Final Notes for Frontend Developer

- Centralize API client configuration: base URL, Accept header, JSON content type, and Bearer token injection.
- Handle 401 globally by clearing the token and redirecting to login.
- Handle 403 separately from 401: user is authenticated but not allowed.
- Render backend validation errors by field using the errors object. Each field maps to an array of messages.
- Do not hardcode category, ingredient, or goal IDs. Always load lookup data from APIs.
- Use pagination meta.pagination to drive page controls, total count, and next/previous UI state.
- Use is_available to display unavailable badges/states; do not remove unavailable items unless the UI specifically filters them.
- For product creation, create the product first, then upload the image using the returned product id.
- Display image.url directly. If image is null, show a placeholder image or empty state.
- For edit screens, fetch product by id to load full relationships and selected IDs before rendering the form.
- For delete actions on categories/ingredients/goals, expect 409 if the item is in use and show a user-friendly message.
- Avoid assuming the Swagger UI is the only source of truth until the product image upload status mismatch is corrected.
- Use loading and empty states for list screens. Product lists can be empty even when lookup data exists.